

CPY Compiler — Project Based Learning Documentation

A custom compiled programming language (.cpy) that features Lexical Analysis → Parsing (AST) → Semantic Check → Bytecode Compilation → Virtual Machine Execution.

Project Division

Part	Student	Assigned Component	Domain
1	Aditya	Virtual Machine + Bytecode I/O	Runtime Execution, Serialization
2	Megha	Semantic Analyzer + Bytecode Compiler	Static Analysis, AST lowering
3	Sameer	Parser (AST Construction)	Recursive Descent Parsing
4	Vridhi	Lexer (Tokenization)	Lexical Analysis

Part 1 — Aditya : Virtual Machine + Bytecode I/O

Files Assigned: * vm/VM.java * compiler/BytecodeWriter.java * compiler/BytecodeReader.java * Main.java

Overview: The Virtual Machine module executes the bytecode instructions generated by the Bytecode Compiler. `BytecodeWriter` serializes these instructions to a binary/textual .cpyc file, and `BytecodeReader` deserializes them back into memory. `Main.java` orchestrates the entire pipeline, exposing the `compile` and `run` commands.

1. The Virtual Machine Architecture (VM.java)

The CPY VM is a **stack-based virtual machine**. Instead of using registers, all mathematical operations, variable loads, and function parameters operate on a Last-In-First-Out (LIFO) stack.

- **stack:** Temporarily holds intermediate values.

- **env**: A hash map acting as the symbol table, associating variable names with their current values.
- **pc**: The Program Counter tracking the index of the next instruction to execute.

```
// vm/VM.java (Architecture Core)
public class VM {
    private final List<Instruction> program;
    private final Stack<Object> stack = new Stack<>();
    private final Map<String, Object> env = new HashMap<>();
    private int pc = 0; // program counter

    public void run() {
        while (pc < program.size()) {
            Instruction instr = program.get(pc);
            execute(instr);
            if (instr.opCode == OpCode.HALT) break;
        }
    }
}
```

Executing Arithmetic (Dynamic Typing) Because CPY is dynamically typed, the VM checks operand types at runtime. For example, ADD supports both numeric addition and string concatenation:

```
case ADD: {
    Object b = stack.pop();
    Object a = stack.pop();
    if (a instanceof Double && b instanceof Double) {
        stack.push((double) a + (double) b);
    } else if (a instanceof String || b instanceof String) {
        stack.push(stringify(a) + stringify(b)); // String concatenation
    } else {
        throw error("ADD requires two numbers or at least one string");
    }
    pc++;
    break;
}
```

Control Flow (Jumping) Instead of linear execution, loops and conditionals alter the pc (Program Counter) directly.

```
case JUMP_IF_FALSE: {
    Object condition = stack.pop();
    if (!isTruthy(condition)) {
        // If false, jump to the given index (operand)
        pc = Integer.parseInt(instr.operand);
    }
}
```

```

    } else {
        // Otherwise, simply proceed to next instruction
        pc++;
    }
    break;
}

```

Array Memory Management (MAKE_ARRAY & ARRAY_LOAD) Because the VM is stack-based, building an array involves popping N elements off the stack, preserving their order, and storing them as a composite List object.

```

// vm/VM.java
case MAKE_ARRAY: {
    int count = Integer.parseInt(instr.operand);
    List<Object> array = new ArrayList<>();

    // Elements were pushed left-to-right (last element is on top)
    Object[] temp = new Object[count];
    for (int i = count - 1; i >= 0; i--) {
        temp[i] = stack.pop(); // Reverse pop to maintain order
    }
    for (Object o : temp) array.add(o);

    stack.push(array); // Push the finalized array reference
    pc++;
    break;
}

case ARRAY_LOAD: {
    Object idxVal = stack.pop(); // E.g., 0
    Object arrVal = stack.pop(); // E.g., [10, 20, 30]
    int idx = toIndex(idxVal);
    List<Object> list = (List<Object>) arrVal;

    stack.push(list.get(idx)); // Pushes 10 back to stack
    pc++;
    break;
}

```

2. Bytecode I/O (BytecodeWriter.java & BytecodeReader.java)

Bytecode serialization allows the compiler to compile once and run indefinitely, without needing to re-parse the source code. The .cpyc file stores instructions sequentially.

```

// compiler/BytecodeWriter.java
public static void write(List<Instruction> instructions, String filename) throws IOException {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {

```

```

writer.write("#CPY_BYTECODE v1.0"); // Header Validation
writer.newLine();

for (Instruction instr : instructions) {
    if (instr.operand == null) {
        writer.write(instr.opCode.name());
    } else if (instr.opCode == OpCode.CONST_STR) {
        // Strings are quoted to preserve whitespace
        writer.write(instr.opCode.name() + " \"" + escapeString(instr.operand) + "\"");
    } else {
        writer.write(instr.opCode.name() + " " + instr.operand);
    }
    writer.newLine();
}
}
}

```

3. Pipeline Orchestration (Main.java)

Main.java combines all compiler passes. It chains the phases from raw source.cpy string down to the generated output .cpyc.

```

// Main.java Pipeline Logic
List<Token> tokens = new Lexer(source).scanTokens(); // 1. Lex
List<Stmt> stmts = new Parser(tokens).parse(); // 2. Parse (AST)
new SemanticAnalyzer().analyze(stmts); // 3. Static Check
List<Instruction> bytecode = new BytecodeCompiler().compile(stmts); // 4. Bytecode Gen
BytecodeWriter.write(bytecode, outFile); // 5. Serialize

```

Part 2 — Megha : Semantic Analyzer + Bytecode Compiler

Files Assigned: * semantic/SemanticAnalyzer.java * compiler/BytecodeCompiler.java
 * compiler/OpCode.java * compiler/Instruction.java

Overview: These files represent the transitional phases of the compiler. The Semantic Analyzer implements static analysis to reject invalid programs before they hit the bytecode generation stage. The Bytecode Compiler lowers the tree structural AST into linear, flat instructions that a simple generic machine can execute.

1. Instruction Set Definition (OpCode.java)

Defines every recognized instruction for the VM processor.

```

public enum OpCode {
    CONST_NUM, CONST_STR, CONST_CHAR, CONST_BOOL, CONST_NULL, // Literals
    ADD, SUB, MUL, DIV, NEG, NOT,                               // Math/Logical
    AND, OR, JUMP, JUMP_IF_FALSE, PRINT,                       // Control/IO
    MAKE_ARRAY, ARRAY_LOAD, ARRAY_STORE,                      // Arrays
    LOAD, STORE, HALT                                         // Registers/Halt
}

```

2. The Semantic Analyzer (SemanticAnalyzer.java)

The semantic analyzer traverses the AST checking for illegal behaviors not captured by grammar. This compiler focuses on the “Use-Before-Definition” and “Duplicate-Declaration” errors.

It uses a single `symbolTable` pass. When a variable goes out of scope or is improperly instantiated, an exception prevents compilation.

```

// semantic/SemanticAnalyzer.java
private void analyzeStmt(Stmt stmt) {
    if (stmt instanceof VarDecl) {
        VarDecl v = (VarDecl) stmt;
        if (symbolTable.containsKey(v.name.lexeme)) { // Duplicate Var validation
            errors.add("Variable '" + v.name.lexeme + "' already declared");
        }
        analyzeExpr(v.initializer);
        symbolTable.put(v.name.lexeme, "any");
    }
    else if (stmt instanceof Assignment) {
        Assignment a = (Assignment) stmt;
        if (!symbolTable.containsKey(a.name.lexeme)) { // Exists validation
            errors.add("Variable '" + a.name.lexeme + "' used before declaration");
        }
        analyzeExpr(a.value);
    }
}
}

```

3. The Bytecode Compiler (BytecodeCompiler.java)

The bytecode compiler uses a technique called **Post-Order AST Traversal**. It generates operands *before* operators, inherently satisfying the stack structure. `1 + 2` generates `[CONST_NUM 1, CONST_NUM 2, ADD]`.

Control Flow: Jump Patching A single-pass compiler doesn’t inherently know where an `else` block ends while currently compiling the `if` block condition. **Jump patching** utilizes placeholders. It logs an instruction index, writes the body, and *patches* the jump statement dynamically later pointing to the size of the program at the body’s end.

```

// compiler/BytecodeCompiler.java
private void compileWhile(WhileStmt stmt) {
    int loopStart = currentIndex(); // Bookmark evaluating the condition

    compileExpr(stmt.condition);
    int jumpExit = emitJump(OpCodes.JUMP_IF_FALSE); // Placeholder for jumping out

    compileStmt(stmt.body);
    emit(OpCodes.JUMP, String.valueOf(loopStart)); // Hop back to condition

    patchJump(jumpExit); // Loop is finished, define jumpExit dest to here
}

private int emitJump(Opcode jumpOp) {
    instructions.add(new Instruction(jumpOp, "0"));
    return instructions.size() - 1; // Return index to be patched later
}

private void patchJump(int instructionIndex) {
    // Modify operand to current end-of-list bounds
    instructions.get(instructionIndex).operand = String.valueOf(instructions.size());
}

```

Expression Compilation (Post-Order Generation) To ensure the Virtual Machine's stack pops the correct variables at runtime, the compiler traverses math expressions recursively left-to-right, emitting the operand *last*.

```

// compiler/BytecodeCompiler.java
private void compileExpr(Expr expr) {
    if (expr instanceof BinaryExpr) {
        BinaryExpr b = (BinaryExpr) expr;

        // 1. Compile the left leaf (pushes to runtime stack)
        compileExpr(b.left);

        // 2. Compile the right leaf (pushes to runtime stack)
        compileExpr(b.right);

        // 3. Emit the operator (pops the last two elements)
        switch (b.operator.type) {
            case PLUS:      emit(OpCodes.ADD); break;
            case MINUS:     emit(OpCodes.SUB); break;
            case STAR:      emit(OpCodes.MUL); break;
            case SLASH:     emit(OpCodes.DIV); break;
            case EQUAL_EQUAL: emit(OpCodes.EQ); break;
            // ...

```

```

    }
  }
}

```

Part 3 — Sameer : Parser (AST Construction)

Files Assigned: * parser/Parser.java * ast/ArrayAccess.java, ArrayAssignment.java, ArrayExpr.java, Assignment.java, BinaryExpr.java, Block.java, Expr.java, ForStmt.java, IfStmt.java, Literal.java, PrintStmt.java, Stmt.java, UnaryExpr.java, VarDecl.java, Variable.java, WhileStmt.java

Overview: Taking the linear sequence of tokens provided by the Lexer, the Parser acts to ensure syntactical correctness and organizes tokens logically into an **Abstract Syntax Tree (AST)**.

1. Abstract Syntax Tree (AST) Files (ast/*.java)

The AST divides logically into two core nodes: * **Stmt** (Statement): Code that performs an action (Variable declaration, Loop, Printing). * **Expr** (Expression): Code that evaluates to a returnable result/value (Literal number, arithmetic evaluation).

For example, a Variable Declaration node maintains a reference to its Name and Initializer Expression:

```

// ast/VarDecl.java
public class VarDecl extends Stmt {
    public final Token name;
    public final Expr initializer;

    public VarDecl(Token name, Expr initializer) {
        this.name = name;
        this.initializer = initializer;
    }
}

```

2. The Parser Implementation (Parser.java)

CPY employs a **Recursive-Descent Parser**. Using methods matched closely to grammatical definitions, it recurses “down” into logical paths matching tokens.

```

// parser/Parser.java - Root Entry
private Stmt statement() {
    if (check(TokenType.LET))    return varDeclaration();
    if (check(TokenType.IF))     return ifStatement();
}

```

```

        if (check(TokenType.WHILE)) return whileStatement();
        if (check(TokenType.FOR))   return forStatement();
        if (check(TokenType.PRINT)) return printStatement();
        if (check(TokenType.LBRACE)) return block();

        return assignmentOrArrayAssignment();
    }

```

Parsing Control Flow (If / While) Parsing control flow requires matching literal syntax (e.g. (and)) and storing the respective blocks into explicit branches. Optional components like `else` are validated gracefully using `match()`.

```

// parser/Parser.java
private Stmt ifStatement() {
    consume(TokenType.IF, "Expected 'if'");
    consume(TokenType.LPAREN, "Expected '(' after 'if'");
    Expr condition = expression();
    consume(TokenType.RPAREN, "Expected ')' after if condition");

    Stmt thenBranch = block();
    Stmt elseBranch = null;

    // The 'else' block doesn't throw an error if it's missing
    if (match(TokenType.ELSE)) {
        elseBranch = block();
    }

    return new IfStmt(condition, thenBranch, elseBranch);
}

```

Expression Parsing (Precedence Climbing) Ensuring strict mathematical logic mandates Precedence order (Expressions matching $1 + 2 * 3$ must evaluate $*$ before $+$). The Parser enforces this strictly by structuring methods cascadingly: `or` > `and` > `equality` > `comparison` > `term` > `factor` > `unary` > `primary`

```

private Expr term() { // Runs Plus / Minus logic
    Expr left = factor(); // Calls mathematical multiplier logic first (*)
    while (match(TokenType.PLUS, TokenType.MINUS)) {
        Token op = previous();
        Expr right = factor();
        left = new BinaryExpr(left, op, right);
    }
    return left;
}

```


Array Literacy Syntax rules explicitly define Array literal configurations.

```
// Literal Array mapping: [expr, expr, ...]
if (match(TokenType.LBRACKET)) {
    List<Expr> elements = new ArrayList<>();
    if (!check(TokenType.RBRACKET)) {
        do {
            elements.add(expression());
        } while (match(TokenType.COMMA));
    }
    consume(TokenType.RBRACKET, "Expected ']' after array literal");
    return new ArrayExpr(elements);
}
```

Part 4 — Vridhi : Lexer (Tokenization)

Files Assigned: *lexer/Lexer.java *lexer/Token.java *lexer/TokenType.java

Overview: Translating raw String source code into analyzable chunks. The Lexer evaluates characters one-by-one, discarding useless constructs (spaces/comments) and structuring syntactical groups into Token objects.

1. Token Definitions (TokenType.java & Token.java)

The dictionary to our compiler system, mapping characters strings to explicit defined enums.

```
// lexer/Token.java
public class Token {
    public final TokenType type; // E.g., NUMBER, LET, FOR, WHILE
    public final String lexeme; // Raw string: "42", "let"
    public final int line; // For syntax error reporting
    // ... Constructor omitted
}
```

2. Lexical Scanning Loops (Lexer.java)

The core maintains two tracking pointers evaluating the array of string text: **start** tracking the beginning character index of current lexeme, and **current** representing the evaluated end index.

```
// lexer/Lexer.java
public List<Token> scanTokens() {
    while (!isAtEnd()) {
        start = current; // Reset token length tracker
        scanToken(); // Analyze string block mapping logical boundaries
    }
}
```

```

        tokens.add(new Token(TokenType.EOF, "", line)); // Guarantee parser halt
        return tokens;
    }

```

Multi-Character Operators Detection Lexifying logic requires lookaheads for combinative constructs like distinguishing `=` (assignment) from `==` (equality assessment).

```

case '=':
    // Pushes pointer +1 forward if next matches '='; outputs EQUAL_EQUAL
    addToken(match('=') ? TokenType.EQUAL_EQUAL : TokenType.EQUAL);
    break;
case '!=':
    addToken(match('=') ? TokenType.BANG_EQUAL : TokenType.NOT);
    break;

```

Context Formatting (Numerics and Ignore Overrides) For `Float` mapping:

```

private void number() {
    while (!isAtEnd() && isDigit(peek())) advance(); // Consume int block

    // Context lookahead for decimals bridging
    if (!isAtEnd() && peek() == '.' && isDigit(peekNext())) {
        advance(); // consume '.'
        while (!isAtEnd() && isDigit(peek())) advance(); // trailing
    }
    addToken(TokenType.NUMBER);
}

```

Keyword Lookups and String Extraction When scanning contiguous alphabetical characters, the Lexer must determine if the word is an arbitrary identifier (like `playerName`) or a reserved keyword (like `while`). It accomplishes this using a cached static map.

```

// lexer/Lexer.java
private void identifier() {
    // Consume characters while alphanumeric
    while (!isAtEnd() && isAlphaNumeric(peek())) advance();

    String text = source.substring(start, current);

    // Check if the exact text exists in the reserved keywords map
    TokenType type = keywords.getOrDefault(text, TokenType.IDENTIFIER);
    addToken(type);
}

```

```

private void string() {
    while (!isAtEnd() && peek() != '"') {
        if (peek() == '\n') line++;
        advance();
    }

    advance(); // Consume the closing quote

    // Emits the exact string without quotation marks
    addToken(TokenType.STRING);
}

```

End-To-End : How the Entire Project Works (Independent Flow)

Understand the CPY compiler as an explicit pipeline mapping Source Text → Bytecode Result Output.

Scenario Code

Assume we provide the compiler the following code string in a .cpy text document:

```

let score = 20 * 5;
print(score);

```

Stage 1: Lexical Analysis (Vridhi's Component)

The raw string passes into `Lexer.java`. Characters are scanned logically and translated. Whitespaces are ignored. **Returns List<Token>:** 1. LET 2. IDENTIFIER("score") 3. EQUAL 4. NUMBER(20) 5. STAR 6. NUMBER(5) 7. SEMICOLON 8. PRINT 9. LPAREN 10. IDENTIFIER("score") 11. RPAREN 12. SEMICOLON 13. EOF

Stage 2: Parser (Sameer's Component)

The array list translates into a Tree structure linking operations matching order-of-operation hierarchies. The Tokens iterate into `Parser.java`, resolving `let` mapping to `varDeclaration()` recursion. **Returns List<Stmt> Base nodes:**
 * **VarDecl Node:** * Name: score * Initializer: BinaryExpr (Literal(20), *, Literal(5)) * **PrintStmt Node:** * Expression: Variable(score)

Stage 3: Semantic Analysis (Megha's Component)

The tree object is recursed linearly through `SemanticAnalyzer.java`, logging local environments. * Parses VarDecl -> Recognizes score doesn't exist -> Adds "score" to internal HashMap. * Parses PrintStmt -> Accesses score

expression -> Validates “score” exists in HashMap. **Result:** Process passes execution (0 errors logged).

Stage 4: Bytecode Compiler (Megha’s Component)

The validated AST recursively collapses into single-step machine instructions, evaluated explicitly Post-Order matching stack structure design. **Returns List<Instruction>:** 1. `CONST_NUM 20` (*Found Literal Left*) 2. `CONST_NUM 5` (*Found Literal Right*) 3. `MUL` (*Resolve AST Binary Node Type*) 4. `STORE "score"` (*Assign Variable Evaluation Node*) 5. `LOAD "score"` (*Print Stmt Load Call Expression*) 6. `PRINT` (*Resolve Evaluate Stmt Form*) 7. `HALT` (*Implicit EOF Resolution*)

Stage 5: Serialization & Virtual Machine Execution (Aditya’s Component)

`BytecodeWriter` writes out the sequence above into a `program.cpyc` document. The interpreter then invokes `VM.java` with the list mapping execution.

Execution Flow: * `PC = 0`: `CONST_NUM 20` -> Machine pushes 20 onto Stack. Array= [20] * `PC = 1`: `CONST_NUM 5` -> Machine pushes 5 onto Stack. Array= [20, 5] * `PC = 2`: `MUL` -> Pops elements (5) & (20). Computes $20 * 5 = 100$. Result pushed onto Stack. Array= [100] * `PC = 3`: `STORE "score"` -> Machine Pops value (100). Sets Internal HashMap Key “score” as 100. Array=[] * `PC = 4`: `LOAD "score"` -> VM Maps HashMap “score”. Returns 100. Array = [100] * `PC = 5`: `PRINT` -> Pops element (100). Stringifies and outputs to IO Console “100”. * `PC = 6`: `HALT` -> Loop breaks.

Application Process Completes Resulting 100 Displayed System Log.

Viva Preparation Guide (Common Questions & Answers)

Since you vibed coded this from scratch, examiners will want to test if you actually understand the core architecture. Here are the most likely questions they will ask based on your specific implementation.

1. “Why did you choose a Stack-Based Virtual Machine instead of a Register-Based one?”

Answer: “A stack-based VM is much easier to compile to and implement. Since operations like `ADD` implicitly pop their operands from the top of the stack and push the result back, I didn’t have to write a complex ‘register allocator’. When compiling an expression like $1 + 2 * 3$, traversing the AST in post-order perfectly aligns with the stack mechanics without needing to track which variables are in which temporary registers.”

2. “How do you handle ‘Order of Operations’ (BODMAS) in your Lexer or Parser?”

Answer: “The lexer doesn’t handle it—it just outputs a flat list of tokens. The **Parser** handles it using a technique called *Recursive Descent with Precedence Climbing*. My parsing methods are structured in a cascading hierarchy from lowest precedence to highest (e.g., **or** calls **and**, which calls **equality**, down to **term** which calls **factor**). This ensures that ***** (multiplication) is evaluated deeper in the AST tree before **+** (addition), naturally enforcing mathematical order of operations.”

3. “What is ‘Jump Patching’ and why did you use it?”

Answer: “My Bytecode Compiler generates instructions in a single pass. When it hits an **if** statement, it needs to emit a **JUMP_IF_FALSE** instruction to jump over the **then** block if the condition is false. But at that moment, it hasn’t compiled the block yet, so it doesn’t know how far to jump. Therefore, it emits a placeholder jump, continues compiling the **then** block, and once finished, it goes back and ‘patches’ that original jump instruction with the exact final index.”

4. “How does your Semantic Analyzer detect undeclared variables?”

Answer: “It runs a single pass over the AST before bytecode compilation. I maintain a **HashMap** called the **symbolTable**. When a variable is declared (**VarDecl**), its name is added to the table. Whenever I hit an **Assignment** or read a **Variable** node, I check if its name exists in that Map. If it isn’t there, it’s being used before declaration, and I throw an error.”

5. “What makes your language ‘Dynamically Typed’?”

Answer: “Types aren’t strictly checked during the parsing phase. Variables don’t have static types (like **int x** or **string y**)—I just use the **let** keyword. The actual type enforcement happens inside the VM at runtime. For example, during the **ADD** operation, the VM checks the Java **instanceof** of the popped stack objects to determine if it should perform a numeric addition or a string concatenation, throwing a runtime error if types are entirely incompatible.”

6. “How did you disambiguate **a = 5 from **a[0] = 5** during parsing?”**

Answer: “Both statements start with an **IDENTIFIER** token. In my **assignmentOrArrayAssignment()** parser method, when the parser consumes an identifier, it peeks at the very next token. If the next token is an **=** sign, it falls into standard **Assignment**. If the next token is a **[** bracket, it knows it’s an **ArrayAssignment**, parses the index expression inside the brackets, and then consumes the **=** sign afterward.”